Assignment 3

Instructions

- Unless specified otherwise,
  - you can use anything in the Python Standard Library.
  - don't use other third-party libraries except for NumPy, SciPy, Pandas, Matplotlib, and Pillow in your code for this assignment.
  - you can assume all user inputs are always valid and require no validation.
- Please follow closely the format of the sample output for each question. Your program should produce **exactly** the same output as the sample (same text, letter case, punctuation, spacing, etc.). The output for each question should end with a single newline character, which has been provided by the `print()` function by default.
- Please follow exactly the specified name, parameter list and return value(s) for the functions to be written if a question has stated (to facilitate our grading). You may define additional (inner) functions for specific tasks or further task decomposition in any question if you see fit.
- **For questions that require you to define specific functions, your main client code (for calling and testing the functions) must be put under an `if __name__ == '__main__'`: check because we may import your script into our test scripts calling your functions in our own ways for grading.**
- **Name your script files as `q1.py` for Question 1, `q2.py` for Question 2, … (case-sensitive). Using other names affects our marking and may result in deduction.**
- You are highly encouraged to annotate your functions to provide type information for parameters and return values, and to include suitable comments in your code to improve the readability. Feel free to modify or extend any comments provided in our starter code if you see fit.
- You should follow the standard PEP8 style guide for writing Python code. Install the `pycodestyle` package via `pip install pycodestyle`. Run the command `pycodestyle qn.py` (*n* = 1, 2, …) to check for any PEP8 style violations, which may lead to minor mark deductions. You should fix all the detected issues before submission.

Question 1 (30%)

The *Collatz conjecture* is a famous unsolved problem in mathematics about a very simple number process:

1. Start with any positive integer n.
2. If n is even, divide it by 2.
3. If n is odd, multiply it by 3 and add 1.
4. Repeat the process with the value derived by last two steps.

The conjecture claims that no matter what positive integer n you start with, the sequence generated by these steps will *always* eventually reach 1. Once the sequence hits 1, it repeats the cycle $1 \to 4 \to 2 \to 1$ forever. Therefore, we stop the process before entering the cycle, and the sequence ends at 1.

The conjecture involves a piecewise function:

$$f(n) = \begin{cases} n/2 & \text{if } n \equiv 0 \pmod 2 \\ 3n+1 & \text{if } n \equiv 1 \pmod 2 \end{cases}$$

It can also be rewritten as a recurrence relation as follows. Let $a_0 = n$ be the starting value, where n is a positive integer. Then define:

$$a_{k+1} = \begin{cases} a_k/2 & \text{if } a_k \equiv 0 \pmod 2 \\ 3a_k+1 & \text{if } a_k \equiv 1 \pmod 2 \end{cases}$$

This recurrence generates the Collatz sequence $a_0, a_1, a_2, \ldots, a_m$, where $a_m = 1$ based on the conjecture.

In a previous tutorial, we examined an iterative approach to generating this sequence. In this assignment, we will implement a recursive solution and use Matplotlib to visualize several statistics of the generated sequences.

According to the piecewise definition, we start with a value n, and $f(n)$ gives the next term in the sequence. Feeding the result $f(n)$ back into the function produces the next term, $f(f(n))$. In recurrence form, each term a_{k+1} in the sequence is derived from the preceding term a_k.

Write a program `q1.py` to implement the functions specified as follows. Download the starter code file q1_starter.py, rename it, and add your implementation.

Implementation

(a) **`collatz(n: int) -> list[int]`**: This function returns a Collatz sequence, which starts from the input value n, generated based on the above piecewise function or recurrence relation.

Notes:
- **Assumption**: You may assume that the input value n is always valid, i.e., a positive integer.
- **Recursion**: This function must be *recursive* – it must call itself. You need to determine what should be returned in the *base case* and in the *recursive case*.
- **Restriction**: You are not allowed to use any loops to implement this function.
- **Return type**: A list of integers representing the full trajectory from n down to 1.

For example, calling `collatz(10)` returns this list: `[10, 5, 16, 8, 4, 2, 1]`.

(b) `plot_collatz_stats(limit: int) -> Figure:` This function analyzes Collatz sequences for all starting integers n from 1 to `limit` (inclusive) and visualizes several statistics using Matplotlib. The function returns a Matplotlib `Figure` object containing four subplots arranged in a 2 × 2 grid.

Notes:
- **Assumption**: You may assume that the input value `limit` is always valid, i.e., a positive integer.

Detailed Steps:

1. **Data Collection**:
   Generate three lists of integer values as follows. Iterate through the range of numbers from 1 to `limit` (inclusive). For each number, call the `collatz()` function to generate its Collatz sequence. Then compute the following statistics for the sequence and append them to their respective lists:
   - **Series Sum**: Calculate the total sum of all terms in the sequence.
   - **Maximum Term**: Find the maximum value (i.e., the highest peak reached) in the sequence.
   - **Total Stopping Time**: Count the number of steps required to reach 1, excluding the starting number. This value is equal to the length of the sequence minus 1.

2. **Subplot Layout Setup**:
   - Create a figure with 2 × 2 subplots (2 rows and 2 columns).
   - Set the figure size as `(12, 10)`, i.e., 12 inches wide and 10 inches tall.

3. **Styling**:
   - Each subplot must have a title, labeled axes, and grid lines for readability.
   - Set `alpha=0.7` to make the plotted objects (the foreground) 30% transparent in all subplots.
   - Show gridlines with dashed line style (`"--"`) and half transparency (`alpha=0.5`) for all subplots.
   - Apply a tight layout using `tight_layout()` to "auto-fit" everything within the figure area without overlapping or being cut off before returning the `Figure` object.

4. **Create Each Subplot**:
   - **Subplot 1 (Top Left)**
     - **Scatter plot: Series Sum vs. Starting Number**
     - Title: "`Series Sum`"
     - x-axis: Starting number n
     - y-axis: Series sum
     - x-label: "`Starting Number (n)`"
     - y-label: "`Sum (Log Scale)`"
     - Use logarithmic scale on the y-axis (to account for large spikes in the data).
     - Marker size (s): 1
     - Mark color (c): "`tab:red`"

   - **Subplot 2 (Top Right)**
     - **Scatter plot: Maximum Reachable Number (Peak)**
     - Title: "`Maximum Reachable Number (Peak)`"
     - x-axis: Starting number n
     - y-axis: Maximum term
     - x-label: "`Starting Number (n)`"
     - y-label: "`Max Value (Log Scale)`"
     - Use logarithmic scale on the y-axis (to account for large spikes in the data).
     - Marker size (s): 1
     - Mark color (c): "`tab:green`"

   - **Subplot 3 (Bottom Left)**
     - **Scatter plot: Total Stopping Time**
     - Title: "`Total Stopping Time`"
     - x-axis: Starting number n
     - y-axis: Number of steps to reach 1
     - x-label: "`Starting Number (n)`"
     - y-label: "`Steps to reach 1`"
     - Marker size (s): 1
     - Mark color (c): "`tab:blue`"

   - **Subplot 4 (Bottom Right)**
     - **Histogram: Distribution of Stopping Times**
     - Title: "`Distribution of Stopping Times`"
     - Plot a histogram showing the frequency distribution of total stopping time values.
     - Use 50 bins for grouping.
     - x-label: "`Stopping Time (Steps)`"
     - y-label: "`Frequency`"
     - Color of the bars (`color`): "`tab:purple`"
     - Edge color of the bars (`edgecolor`): "`black`"

Sample Client Code

```
# Sample client
if __name__ == "__main__":
    # Print some Collatz sequences and their statistics
    for n in [1, 9, 10]:
        s = collatz(n)
        print(f"n: {n}")
        print(f"Sequence: {s}")
        print(f"Sum: {sum(s)}, Max: {max(s)}, Stopping time: {len(s) - 1}")
```

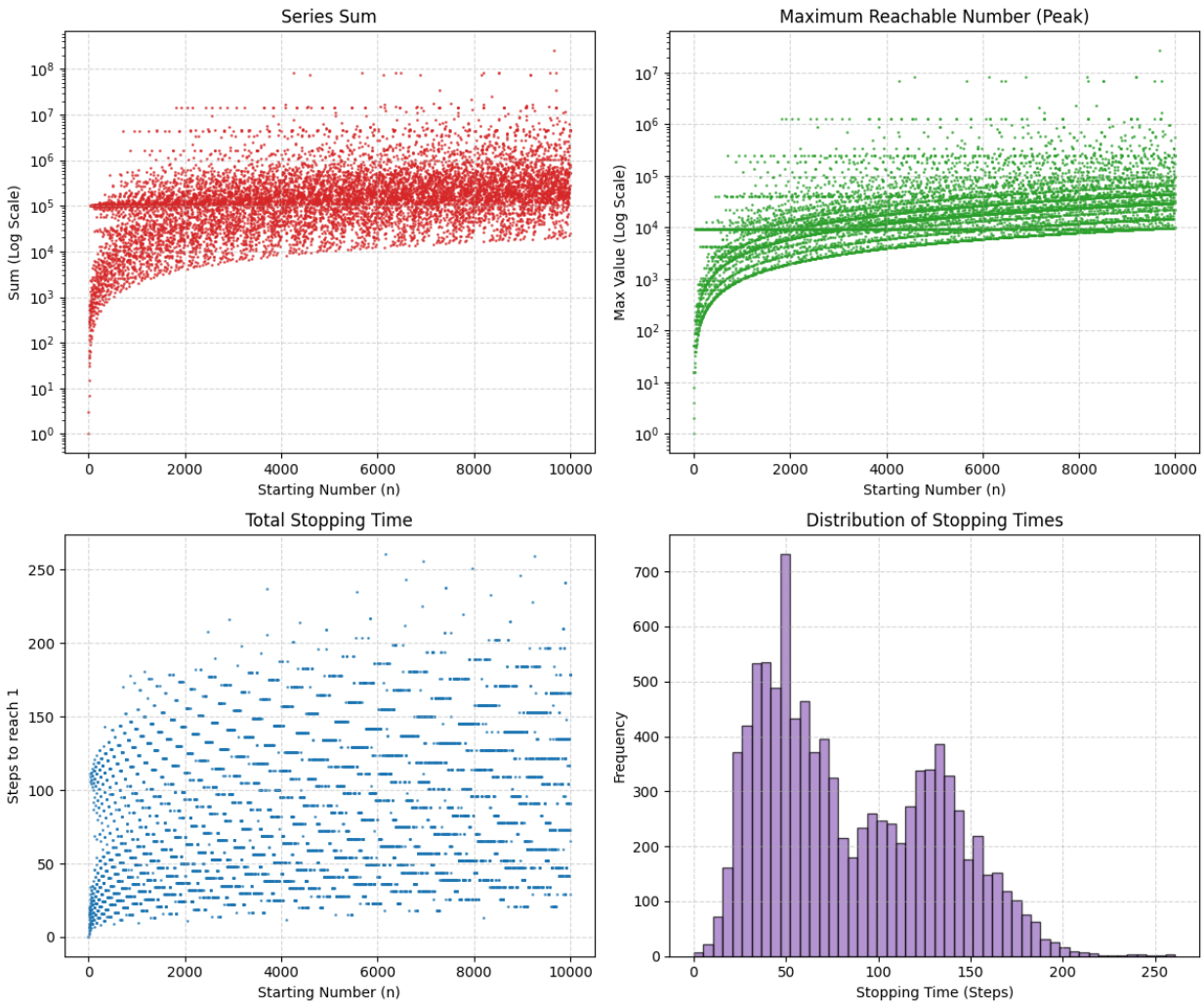
```
    # Create some plots of the Collatz sequence analysis
    fig1 = plot_collatz_stats(limit=10000)
    fig1.savefig('collatz_stats1.png')
    fig2 = plot_collatz_stats(limit=30000)
    fig2.savefig('collatz_stats2.png')
```

Sample Output

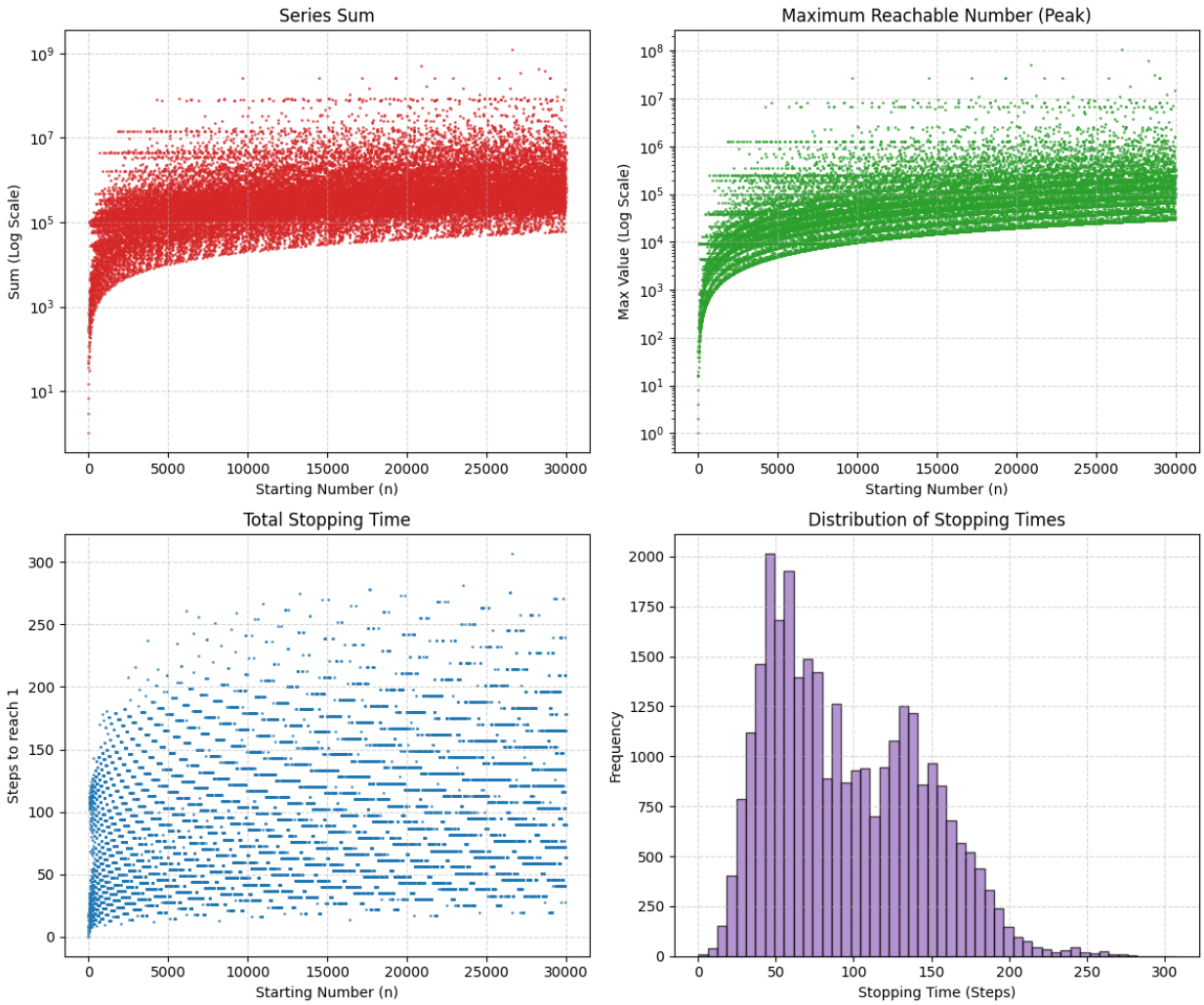
```
n: 1
Sequence: [1]
Sum: 1, Max: 1, Stopping time: 0
n: 9
Sequence: [9, 28, 14, 7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1]
Sum: 339, Max: 52, Stopping time: 19
n: 10
Sequence: [10, 5, 16, 8, 4, 2, 1]
Sum: 46, Max: 16, Stopping time: 6
```

Apart from the above textual output, the sample client code saves the generated figures as PNG files. You may compare your results with the sample images we provided (see below screenshots) for your testing and debugging. Your output images should match the samples as closely as possible — ideally exactly — including all the stated styling requirements.

For starting numbers (n) from 1 to 10,000:



For starting numbers (n) from 1 to 30,000:



If you visit the Wikipedia page for the Collatz conjecture, you will see that our scatter plots and histograms of total stopping times closely resemble the figures shown there.

Hint: Use `Axes.scatter()` for the first three subplots and `Axes.hist()` for the final subplot. Refer to the official documentation if needed. Note that the state-based interface functions `pyplot.scatter()` and `pyplot.hist()` are harder to use in this case because they provide less explicit control over multiple subplots.

(For your reference, these figures are generated by Matplotlib 3.10.8. Other versions should also work.)

Question 2 (30%)

In this question, we will experiment with basic image processing using NumPy and Pillow. The tasks include using the Pillow package to load an image file as a NumPy array and manipulating the array to achieve some visual effects.

Write a Python script `q2.py` that applies a circular, feathered (blurred) transparency mask to any image. The program should take an input image file, load it, and output the same image cropped using a circular mask with feathered edges (i.e., a soft-edge vignette mask), as shown in Figure 1. Refer to our provided starter code (`q2_starter.py`), sample input and output image files for your coding and testing.

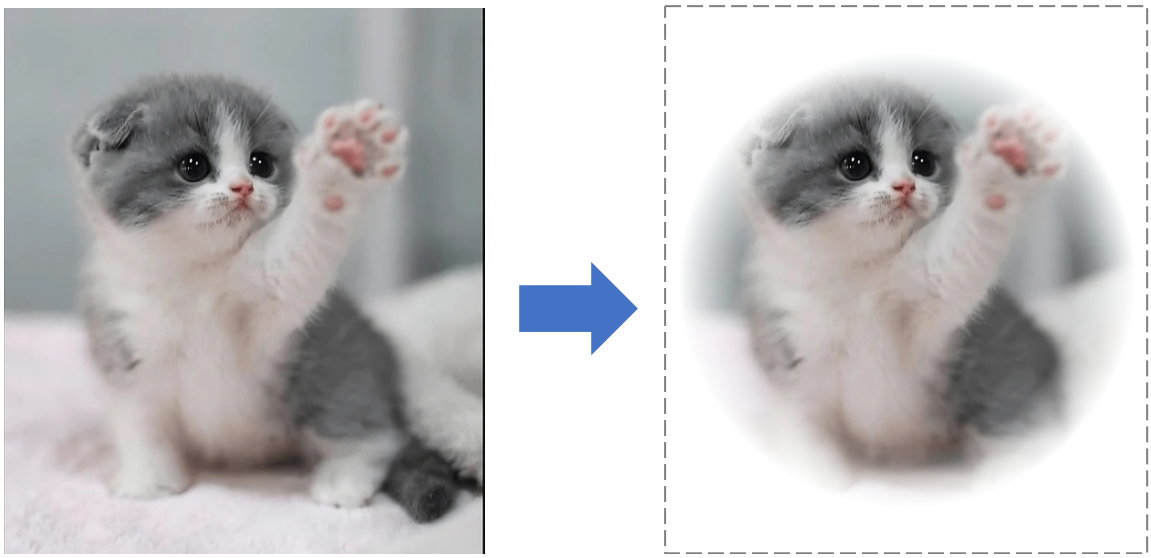


Figure 1: Input image (left) and output image (right) of the program (image source: shutterstock.com)

Background

In image processing, a *mask* is a data structure that defines which parts of an image are visible. It is typically a single-channel (grayscale) image used to control the transparency of another image. A mask has the same dimensions as the target image to which it is applied. It does not contain color information — only intensity values.

A *hard mask* uses binary values (either 0 or 1 to represent that the pixel is "ignored" or "selected"), while a *soft mask* uses a continuous range of values (i.e., a gradient, for example, from 0.0 to 1.0) to create smooth transitions or "feathering," which prevents the edges from appearing jagged. For instance, 0.0 means fully transparent (invisible), 1.0 means fully opaque (solid), 0.5 means the pixel is 50% transparent, and 0.85 means it is mostly opaque but slightly see-through.

Most digital images use the RGB model (Red, Green, Blue). To support transparency, a fourth channel — called the *Alpha channel* — is added, resulting in the RGBA model. This additional channel stores the mask data. When you "apply a mask" to an image to make it partially transparent, you are effectively taking your 2D grayscale mask and plugging it into that 4th slot (the Alpha channel) of the image data.

Not all file formats store transparency information. For example, JPEG does not support an Alpha channel. If you apply a circular mask to a photo and save it as a JPEG file, the mask is discarded, and the transparent areas usually turn solid white or black. In contrast, PNG (Portable Network Graphics) supports the Alpha channel and uses "lossless" compression, meaning the exact mask values (including smooth gradients for soft edges) are preserved. Most image data is commonly stored using 8-bit integers (`uint8`) per channel. When mapping a continuous 0.0–1.0 transparency range to the Alpha channel of an 8-bit PNG image, the values are scaled to an integer range from 0 to 255.

Algorithmic Overview

Unlike a simple crop, a soft-edge vignette uses a gradient of transparency. Each pixel is given a mask or transparency value based on its position in the input image, i.e., its (y, x) coordinates in a rectangular grid system derived from the image. **Note**: while we usually think of coordinates (x, y) in Cartesian math as horizontal then vertical, we access pixel data in NumPy arrays as `image[row, col]`, which corresponds to (y, x) — vertical then horizontal. The program includes the following core steps:

1. **Coordinate System Generation**

   The algorithm begins by defining the "canvas." Since images are 2D grids, we use NumPy to generate a coordinate system where every pixel is assigned an (y, x) value.
   - **Logic**: Instead of looping, we create two matrices: one where every row is $[0, 1, 2, \ldots, W]$ and one where every column is $[0, 1, 2, \ldots, H]$ for an image of width W and height H.
   - **Goal**: To have a reference point (or address) for every single pixel, so we can calculate its position relative to the center of the image.

2. **Radial Distance Calculation**

   Once we have the coordinates, we calculate how far each pixel is from the center of the image.
   - **Logic**: The distance d of any pixel (y, x) from the center (y_c, x_c) is calculated using the Euclidean distance formula.
   - **Result**: A 2D array (with the same size as the image) where the center value is 0 (or close to 0), and the values increase as you move toward the edges or corners.

3. **Normalized Linear Interpolation (The Mask)**

   The distances are transformed into a normalized soft mask (with *alpha* values between 0.0 and 1.0). We define two boundaries: the $\underline{inner\ radius}$ (where the fade starts) and the $\underline{outer\ radius}$ (where the image fully disappears).
   - **Logic**: For any pixel at distance d,
     - If $d \leq inner\ radius$, $Alpha = 1.0$ (Opaque).
     - If $d \geq outer\ radius$, $Alpha = 0.0$ (Transparent).
     - If $inner\ radius < d < outer\ radius$, reduce $Alpha$ linearly as d increases.

4. **Alpha Blending and Channel Stacking**

   The final step is to multiply the 0.0–1.0 mask by 255 to match the standard 8-bit image format and attach this 1-channel mask as the Alpha Channel of the original 3-channel RGB image. So, the result is a 4-channel RGBA array. When saved as a PNG, the image viewer reads this 4th layer and renders the pixels with varying levels of transparency against whatever background is behind the image.

Summary of Data Flow

Input image (Pixel data) → Grid (Spatial data)
→ Distances (Geometric data)
→ Mask (Transparency data)
→ RGBA Image (Final output)

Implementation

(a) **`get_radial_distances(shape: tuple[int, ...]) -> np.ndarray`**: Create a 2D map to represent how far each pixel is from the image center. This function implements the first two core steps in the above algorithm.

- **Detailed Steps:**
  1. Get the image's height (h) and width (w) from the input shape tuple.
  2. Calculate the center of the image as: $y_c = h/2$, $x_c = w/2$.

3. Generate two coordinate matrices, namely y and x, to store the y- and x-coordinates of each pixel in the image.

   Hint: instead of writing explicit loops, you may use the `np.meshgrid()` function to generate two coordinate matrices for the y and x axes. For example, for a very small image (such as our sample image *"heart1.jpg"*) that is 7 pixels tall and 9 pixels wide (i.e., h = 7 and w = 9), the following code snippet:

```
cols = np.arange(w)
rows = np.arange(h)
x, y = np.meshgrid(cols, rows)
```

   will generate the matrices x and y with the below content:

x:	y:
[[0 1 2 3 4 5 6 7 8] [0 1 2 3 4 5 6 7 8] [0 1 2 3 4 5 6 7 8] [0 1 2 3 4 5 6 7 8] [0 1 2 3 4 5 6 7 8] [0 1 2 3 4 5 6 7 8] [0 1 2 3 4 5 6 7 8]]	[[0 0 0 0 0 0 0 0 0] [1 1 1 1 1 1 1 1 1] [2 2 2 2 2 2 2 2 2] [3 3 3 3 3 3 3 3 3] [4 4 4 4 4 4 4 4 4] [5 5 5 5 5 5 5 5 5] [6 6 6 6 6 6 6 6 6]]

   When pairing up the elements of these two arrays, each tuple will represent the coordinates of a pixel, e.g., $(6, 0)$ addresses the bottom-left corner whereas $(0, 8)$ the top-right corner.

   Alternatively, you may use `np.ogrid` (short for Open Grid) instead of `np.meshgrid()` to get sparse coordinate vectors. Note that `np.ogrid` is a class instance (not a function) that uses slicing syntax, i.e., square brackets [ ] instead of parentheses ( ), to generate the vectors in 2D-array format. For example, if h = 7 and w = 9, the following code snippet:

```
y, x = np.ogrid[:h, :w]
```

   will generate the "vectors" x and y with the below content:

x:	y:
[[0 1 2 3 4 5 6 7 8]]	[[0] [1] [2] [3] [4] [5] [6]]

   Utilizing sparse vectors is more space-efficient than dense grids. They can be used to derive coordinates and compute the distances seamlessly in the next step, thanks to *broadcasting*.

4. Calculate the Euclidean distance for every pixel (y, x) from image center (y_c, x_c) using the formula:

$$d = \sqrt{(x - x_c)^2 + (y - y_c)^2}$$

   Note: You should not use loops to repeat the calculation for each pixel here. When x and y are arrays, then d will also be an array. Return this array as this function's return value.

   For the sample image *"heart1.jpg"*, the center is calculated as (3.5, 4.5) and the computed distance array (rounded to 3 decimal places) is as shown below (the center is underlined):

```
[[5.701 4.95  4.301 3.808 3.536 3.536 3.808 4.301 4.95 ]
 [5.148 4.301 3.536 2.915 2.55  2.55  2.915 3.536 4.301]
 [4.743 3.808 2.915 2.121 1.581 1.581 2.121 2.915 3.808]
 [4.528 3.536 2.55  1.581 0.707 0.707 1.581 2.55  3.536]
 [4.528 3.536 2.55  1.581 0.707 0.707 1.581 2.55  3.536]
 [4.743 3.808 2.915 2.121 1.581 1.581 2.121 2.915 3.808]
 [5.148 4.301 3.536 2.915 2.55  2.55  2.915 3.536 4.301]]
```

(b) **`create_feathered_mask(dist: np.ndarray, inner_radius: int, outer_radius: int) -> np.ndarray`**: Convert the distances (computed in part (a)) into a normalized transparency map (soft mask). This function implements the core step 3 in the algorithm.

- **Detailed Steps:**
  Map the distance values to a range of 0.0 to 1.0 (normalized distances) using **linear interpolation**.
  - **Parameters**:
    - $inner_radius$: The distance from the center where the image is 100% opaque.
    - $outer_radius$: The distance where the image becomes 100% transparent.
  - **Logic**: For any pixel at distance d (each element in the `dist` array),
    - If $d \leq inner_radius$, $Alpha = 1.0$ (fully opaque).
    - If $d \geq outer_radius$, $Alpha = 0.0$ (fully transparent).
    - If $inner_radius < d < outer_radius$, calculate $Alpha$ using **linear interpolation**:

$$normalized_dist = \frac{d - inner_radius}{outer_radius - inner_radius}$$

$$Alpha = 1 - normalized_dist$$

- **Hints**: Again, we can treat d as an array instead of just a single distance value in the formula to avoid the need of looping. Since d is an array, $normalized_dist$ and hence $Alpha$ will also be arrays. Moreover, to simplify the multiway branching logic (optional), you may use the `np.clip()` function to keep all alpha values calculated from the above formulae strictly within the $[0.0, 1.0]$ range. This is called *clipping*, which means that values less than 0 will be set to 0 and values greater than 1 will be set to 1.

- **Return value**: the $Alpha$ array (mask)

  For the sample image *"heart1.jpg"*, the $Alpha$ array (rounded) before clipping is like this:

```
[[-2.715 -1.857 -1.116 -0.552 -0.241 -0.241 -0.552 -1.116 -1.857]
 [-2.083 -1.116 -0.241  0.468  0.886  0.886  0.468 -0.241 -1.116]
 [-1.621 -0.552  0.468  1.376  1.993  1.993  1.376  0.468 -0.552]
 [-1.375 -0.241  0.886  1.993  2.992  2.992  1.993  0.886 -0.241]
 [-1.375 -0.241  0.886  1.993  2.992  2.992  1.993  0.886 -0.241]
 [-1.621 -0.552  0.468  1.376  1.993  1.993  1.376  0.468 -0.552]
 [-2.083 -1.116 -0.241  0.468  0.886  0.886  0.468 -0.241 -1.116]]
```

  After clipping, the $Alpha$ array (rounded) or the clipped mask is like this:

```
[[0.    0.    0.    0.    0.    0.    0.    0.    0.   ]
 [0.    0.    0.    0.468 0.886 0.886 0.468 0.    0.   ]
 [0.    0.    0.468 1.    1.    1.    1.    0.468 0.   ]
 [0.    0.    0.886 1.    1.    1.    1.    0.886 0.   ]
 [0.    0.    0.886 1.    1.    1.    1.    0.886 0.   ]
 [0.    0.    0.468 1.    1.    1.    1.    0.468 0.   ]
 [0.    0.    0.    0.468 0.886 0.886 0.468 0.    0.   ]]
```

(c) **`apply_alpha_mask(image: np.ndarray, mask: np.ndarray) -> np.ndarray`**: Attach the gradient mask to the input image as a functional Alpha channel. This function implements the core step 4 in the algorithm. The output is the masked image array.

- **Assumptions**: The input `image` array is always 3D; its 3rd dimension has either 3 or 4 channels.
- **Detailed Steps:**
  1. **Channel Augmentation**: If the input is a 3-channel image (RGB), augment it with a 4th channel. **Hint**: One way to do this is using the `np.dstack()` function to "stack" a new alpha layer onto the original RGB data. For example, the following code creates a new 3D array (`image_4ch`) with shape (h, w, 4) if `image_3ch` has shape (h, w, 3) and `alpha` has shape (h, w):

     ```
     image_4ch = np.dstack([image_3ch, alpha])
     ```

     After this step, the image is guaranteed to have 4 channels (RGBA). Note that the contents of the `alpha` array are not important at this stage, since the 4th channel will be replaced in the next step. Therefore, its elements may be initialized to 0 or 255 or left empty.

  2. **Scaling:** Scale the 0.0–1.0 mask values to 0–255 by multiplying `mask` with 255.
  3. **Channel replacement**: Replace the 4th channel (regardless of whether it originally existed in the input image or was added in a previous step) with the converted mask.
  4. **Casting:** The final array returned by this function must be cast from floating-point values to 8-bit unsigned integers (`np.uint8`) to ensure compatibility with image-saving libraries.

- **Note:** Ensure that this function does not modify the original image and mask arrays.
- **Hints**: Use `np.ndarray.copy()` to copy from any of the arrays whenever it fits.

For the sample image *"heart1.jpg"*, the scaled mask (rounded) is like this:

```
[[  0.      0.      0.      0.      0.      0.      0.      0.      0.   ]
 [  0.      0.      0.    119.347 226.    226.    119.347   0.      0.   ]
 [  0.      0.    119.347 255.    255.    255.    255.    119.347   0.   ]
 [  0.      0.    226.    255.    255.    255.    255.    226.      0.   ]
 [  0.      0.    226.    255.    255.    255.    255.    226.      0.   ]
 [  0.      0.    119.347 255.    255.    255.    255.    119.347   0.   ]
 [  0.      0.      0.    119.347 226.    226.    119.347   0.      0.   ]]
```

After the casting operation, it will become like this:

```
[[  0   0   0   0   0   0   0   0   0]
 [  0   0   0 119 226 226 119   0   0]
 [  0   0 119 255 255 255 255 119   0]
 [  0   0 226 255 255 255 255 226   0]
 [  0   0 226 255 255 255 255 226   0]
 [  0   0 119 255 255 255 255 119   0]
 [  0   0   0 119 226 226 119   0   0]]
```

Restrictions

For each of the required functions, your implementation must follow these restrictions:

1. **No loops**: You are not allowed to use `for` or `while` loops to iterate over pixels. You must use NumPy vectorized operations for simpler coding and efficiency.
2. **Pure functions**: The required functions must be implemented as *pure* functions.

The `apply_vignette()` function provided in the starter code implements the soft-edge vignette effect using function composition, meaning it forms a *pipeline* that calls the required functions.

Sample Client Code

```
def apply_vignette(image_path: str, out_path: str, inner_ratio: float = 0.7,
                   outer_ratio: float = 0.95) -> None:
    # Load the input image file
    try:
        print("Input path:  ", image_path)
        input_image = Image.open(image_path)
    except FileNotFoundError as e:
        print(e)
        return None  # Skip image processing

    # Convert to RGBA if it already has transparency/alpha info
    if input_image.mode in ("RGBA", "LA", "P"):
        input_image = input_image.convert("RGBA")  # 4 channels
    else:
        input_image = input_image.convert("RGB")   # 3 channels

    # Convert the input image to a NumPy array
    image = np.array(input_image, dtype=np.float32)
    print("Input shape: ", image.shape)

    # Find the maximum radius, which is half of min(height, width)
    max_radius = min(image.shape[:2]) / 2
    inner_radius = max_radius * inner_ratio
    outer_radius = max_radius * outer_ratio

    # Apply the mask to the image array
    distances = get_radial_distances(image.shape)
    mask = create_feathered_mask(distances, inner_radius, outer_radius)
    masked_image = apply_alpha_mask(image, mask)

    # Save the output array as a PNG file
    Image.fromarray(masked_image).save(out_path, "PNG")
    print("Output shape:", masked_image.shape)
    print("Output path: ", out_path)

# Sample client
if __name__ == "__main__":
    apply_vignette("cat1.jpg", "cat1_cropped.png")
    apply_vignette("cat2.jpg", "cat2_cropped.png")
    apply_vignette("cat3.png", "cat3_cropped.png")
    apply_vignette("cat4.png", "cat4_cropped.png")
    apply_vignette("heart1.jpg", "heart1_cropped.png")
    apply_vignette("heart2.jpg", "heart2_cropped.png")
```

Sample Output

```
Input path:   cat1.jpg
Input shape:  (600, 528, 3)
Output shape: (600, 528, 4)
Output path:  cat1_cropped.png
Input path:   cat2.jpg
Input shape:  (400, 654, 3)
Output shape: (400, 654, 4)
Output path:  cat2_cropped.png
Input path:   cat3.png
Input shape:  (628, 800, 4)
Output shape: (628, 800, 4)
Output path:  cat3_cropped.png
```

```
Input path:   cat4.png
Input shape:  (800, 800, 4)
Output shape: (800, 800, 4)
Output path:  cat4_cropped.png
Input path:   heart1.jpg
Input shape:  (7, 9, 3)
Output shape: (7, 9, 4)
Output path:  heart1_cropped.png
Input path:   heart2.jpg
Input shape:  (8, 10, 3)
Output shape: (8, 10, 4)
Output path:  heart2_cropped.png
```

Apart from the above textual output, the sample client code saves the masked image arrays as PNG files named "`<input image filename>_cropped.png`". The small images *"heart1.jpg"* and *"heart2.jpg"* are provided to facilitate unit testing and debugging of the required functions.

Below are a screenshot of the first four sample output images:

Question 3 (40%)

In this question, you will create a plot of demographic data for Hong Kong. It is interesting to examine whether our city has a higher male or female population. The question becomes more complex when age groups are taken into account. To study this issue, a *population pyramid* is commonly used to provide a graphical illustration of the distribution of a population by age group and gender.

The dataset [URL] used in this question is obtained from the Census and Statistics Department of the Hong Kong SAR Government. The source provides the data in various file formats; for this task, we use the CSV format (excluding symbols in the data cells). The original filename is "*Table 110-01002 (excl symbols).csv*". For your convenience, we have uploaded a copy of this file to Blackboard and renamed it "*q3_data.csv*". You may download the data file and from Blackboard for your work.

You may visit the data source website to get an idea of the structure of the raw data, as shown in the screenshot (Figure 2) below:

		Population															
		Number ('000)								% of total population							
Year		2022		2023		2024		2025		2022		2023		2024		2025	
Reference time-point		Mid-year	Year-end	Mid-year	Year-end	Mid-year	Year-end	Mid-year	Year-end p	Mid-year	Year-end	Mid-year	Year-end	Mid-year	Year-end	Mid-year	Year-end p
Sex	Age																
Male	0	17.8	16.7	17.0	17.3	17.4	18.1	16.7	15.1	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
	1	19.2	18.8	18.3	17.4	17.4	17.3	17.2	16.9	0.3	0.3	0.2	0.2	0.2	0.2	0.2	0.2
	2	23.3	20.8	19.6	19.2	18.5	17.6	17.6	17.2	0.3	0.3	0.3	0.3	0.2	0.2	0.2	0.2
	3	24.3	24.7	24.1	21.4	20.0	19.3	18.7	17.7	0.3	0.3	0.3	0.3	0.3	0.3	0.2	0.2
...																	
	83	15.3	14.2	15.3	15.6	13.8	12.6	13.5	14.5	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
	84	14.9	15.3	14.4	13.4	14.4	14.7	12.9	11.7	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
	≥85	86.5	89.3	89.6	94.2	94.0	97.9	95.5	100.1	1.2	1.2	1.2	1.3	1.2	1.3	1.3	1.3
	All age	**3 354.5**	**3 409.8**	**3 435.2**	**3 425.3**	**3 417.9**	**3 404.5**	**3 400.1**	**3 404.4**	**45.7**	**45.6**	**45.6**	**45.5**	**45.4**	**45.4**	**45.3**	**45.3**
Female	0	16.8	15.6	15.7	15.8	15.8	16.5	15.4	14.0	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
	1	18.3	18.0	17.1	16.0	16.2	15.8	15.5	15.5	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
	2	21.7	19.7	18.6	18.3	17.5	16.2	16.2	15.7	0.3	0.3	0.2	0.2	0.2	0.2	0.2	0.2
	3	23.0	23.1	22.4	20.2	18.9	18.5	17.7	16.2	0.3	0.3	0.3	0.3	0.3	0.2	0.2	0.2
...																	
	83	16.6	15.0	16.0	16.3	14.6	13.8	14.9	15.8	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
	84	17.5	17.5	16.1	14.4	15.4	15.7	13.8	13.1	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
	≥85	143.5	145.9	146.3	150.0	149.2	151.8	149.7	152.9	2.0	2.0	1.9	2.0	2.0	2.0	2.0	2.0
	All age	**3 991.6**	**4 062.8**	**4 100.9**	**4 102.6**	**4 106.2**	**4 096.1**	**4 098.8**	**4 106.4**	**54.3**	**54.4**	**54.4**	**54.5**	**54.6**	**54.6**	**54.7**	**54.7**
Both sexes	0	**34.6**	**32.3**	**32.7**	**33.1**	**33.2**	**34.6**	**32.1**	**29.1**	**0.5**	**0.4**	**0.4**	**0.4**	**0.4**	**0.5**	**0.4**	**0.4**
	1	**37.5**	**36.8**	**35.4**	**33.4**	**33.6**	**33.1**	**32.7**	**32.4**	**0.5**	**0.5**	**0.5**	**0.4**	**0.4**	**0.4**	**0.4**	**0.4**
	2	**45.0**	**40.5**	**38.2**	**37.5**	**36.0**	**33.8**	**33.8**	**32.9**	**0.6**	**0.5**	**0.5**	**0.5**	**0.5**	**0.5**	**0.5**	**0.4**
	3	**47.3**	**47.8**	**46.5**	**41.6**	**38.9**	**37.8**	**36.4**	**33.9**	**0.6**	**0.6**	**0.6**	**0.6**	**0.5**	**0.5**	**0.5**	**0.5**
...																	
	83	**31.9**	**29.2**	**31.3**	**31.9**	**28.4**	**26.4**	**28.4**	**30.3**	**0.4**	**0.4**	**0.4**	**0.4**	**0.4**	**0.4**	**0.4**	**0.4**
	84	**32.4**	**32.8**	**30.5**	**27.8**	**29.8**	**30.4**	**26.7**	**24.8**	**0.4**	**0.4**	**0.4**	**0.4**	**0.4**	**0.4**	**0.4**	**0.3**
	≥85	**230.0**	**235.2**	**235.9**	**244.2**	**243.2**	**249.7**	**245.2**	**253.0**	**3.1**	**3.1**	**3.1**	**3.2**	**3.2**	**3.3**	**3.3**	**3.4**
	All age	**7 346.1**	**7 472.6**	**7 536.1**	**7 527.9**	**7 524.1**	**7 500.6**	**7 498.9**	**7 510.8**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**

Figure 2: Raw data table provided by Hong Kong Census and Statistics Department

The raw data structure cannot be plotted directly as the desired chart. Therefore, data transformation is required, and Pandas can help with this task.

You will complete a program `q3.py` that reads the Hong Kong population data and generates a population pyramid—a back-to-back horizontal bar chart comparing male and female demographics across different age groups. An example figure is shown in Figure 3.

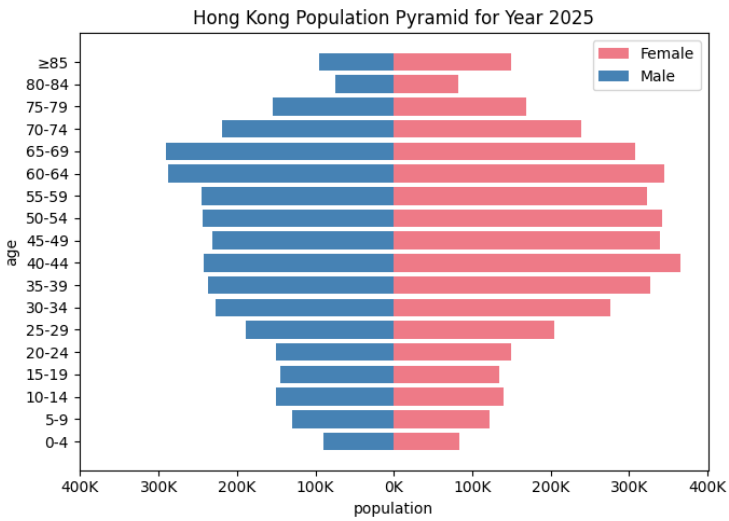


Figure 3: An example population pyramid (comparing male vs. female per age group) created by the program

Download the starter code file `q3_starter.py`, rename it, and add your implementation. The starter code provides a function named `read_population_data()`. It reads population data in raw format from the CSV file, extracts the mid-year columns from the left half of the data table (i.e., those Number ('000) columns), and returns a `DataFrame` with a list of available years (in `str` format). The returned objects are displayed as follows:

Raw population `DataFrame`:

```
        Sex  Age   2022   2023   2024   2025
0      Male    0   17.8   17.0   17.4   16.7
1      Male    1   19.2   18.3   17.4   17.2
2      Male    2   23.3   19.6   18.5   17.6
3      Male    3   24.3   24.1   20.0   18.7
4      Male    4   24.8   25.1   24.4   20.1
..      ...  ...    ...    ...    ...    ...
167  Female   81   15.4   16.0   17.7   17.9
168  Female   82   16.5   15.1   15.6   16.9
169  Female   83   16.6   16.0   14.6   14.9
170  Female   84   17.5   16.1   15.4   13.8
171  Female  ≥85  143.5  146.3  149.2  149.7

[172 rows x 6 columns]
```

List of year strings:

```
['2022', '2023', '2024', '2025']
```

Implementation

(a) **`transform_data(pdf: pd.DataFrame, year: str, k: int = 5) -> pd.DataFrame`**: Transform the raw population data into a consolidated format suitable for plotting, grouping ages by k rows and reorganizing the data values into "Male" and "Female" columns side by side.

Function Arguments:

- `pdf` (`pd.DataFrame`): the population data in original (raw) format
- `year` (`str`): a string representing the year of the data column to retrieve
- k (`int`): age group interval (i.e., group by every k rows)

The raw data returned by `read_population_data()` contains individual rows for each age and sex. To plot a population pyramid, we need to restructure this into a single `DataFrame` where "Male" and "Female" are side-by-side columns, and ages are grouped into intervals of k years.

To simplify the restructuring process, we have predefined several (inner) helper functions:

- `filter_by_sex(df: pd.DataFrame, sex: str) -> pd.DataFrame`:
  Returns a new `DataFrame` containing only the rows where the 'Sex' column matches the gender specified by the `sex` string (e.g., "Male").
- `select_age_year(df: pd.DataFrame) -> pd.DataFrame`:
  Filters the columns of the `DataFrame`, keeping only the 'Age' column and the column labeled by the year specified by the outer function parameter `year`.
- `group_by_age(df: pd.DataFrame) -> pd.DataFrame`:
  Groups individual age rows into bins (e.g., grouping ages from 0 to 4 into a bin labeled "0-4") by dividing the `DataFrame` index by k. It utilizes an inner helper function `concat_first_last()` to generate the string labels for the bins while simultaneously summing the population counts for the selected year column. This aggregates the individual rows into age groups, making the data suitable for the pyramid chart.

  Note: `concat_first_last(group: pd.Series) -> str`: This is a helper for the aggregation. It joins the first and last age in the group with a hyphen (e.g., "0-4"). If the group has only one row, it simply returns that age (e.g., "≥85", which is the only case in this dataset).
- `rename_year(df: pd.DataFrame, sex: str) -> pd.DataFrame`:
  Returns a `DataFrame` with the population column renamed from a generic year (e.g., '2025') to the specific gender (e.g., 'Male'). This helps avoid column name conflicts in a later step of merging the `DataFrame` objects for the two genders.

Detailed Steps:

- **Filter rows**: Use the `filter_by_sex()` helper function to derive two subset `DataFrame` objects for the two genders: 'Male' and 'Female'.
- **Filter columns**: Use `select_age_year()` to retain only the 'Age' column and specified year column for each subset `DataFrame`.
- **Group by age**: Then apply `group_by_age()` to each subset `DataFrame`. For example, when k = 5, the male `DataFrame` will look something like the following:

```
      Age   2025
0     0-4   90.3
1     5-9  129.2
2   10-14  149.6
3   15-19  144.3
4   20-24  150.7
5   25-29  188.6
6   30-34  226.8
7   35-39  237.0
8   40-44  241.8
9   45-49  231.6
10  50-54  244.2
11  55-59  244.6
12  60-64  288.1
13  65-69  290.1
14  70-74  219.0
15  75-79  154.7
16  80-84   74.0
17    ≥85   95.5
```

- **Reset index**: Ensure the female `DataFrame` index is reset to start from 0. This is crucial because the grouping logic depends on the row index. If you neglect this step, the age groups resulting from the male and female grouping operations may differ. Consequently, the two `DataFrame` objects cannot be merged easily in the final step.
- **Rename column**: Next, use `rename_year()` to rename the specified year column to 'Male' in the male `DataFrame` and to 'Female' in the female `DataFrame`.
- **Merging**: Finally, concatenate them into a single `DataFrame` using the `pd.concat()` function. The result will look like the following:

```
      Age   Male  Female
0     0-4   90.3    83.8
1     5-9  129.2   121.7
2   10-14  149.6   140.2
3   15-19  144.3   134.2
4   20-24  150.7   149.6
5   25-29  188.6   204.3
6   30-34  226.8   276.4
7   35-39  237.0   326.6
8   40-44  241.8   364.9
9   45-49  231.6   339.5
10  50-54  244.2   341.7
11  55-59  244.6   322.7
12  60-64  288.1   344.9
13  65-69  290.1   308.1
14  70-74  219.0   239.3
15  75-79  154.7   168.6
16  80-84   74.0    82.6
17    ≥85   95.5   149.7
```

Hint: Instead of duplicating the operations for both genders, you may use `map()` to apply filtering, selection, grouping, and renaming consistently to both gender `DataFrame` objects.

Using the provided helper functions is optional. You may choose a different approach, provided that your code produces the required consolidated `DataFrame` for our grading using function-based unit tests and for the next step (charting).

(b) **`population_pyramid(pdf: pd.DataFrame, city: str, year: str, k: int = 5, w: int = 6, h: int = 4) -> Figure`**: Creates a Matplotlib figure showing a population pyramid, using horizontal bars (barh) to compare male and female demographics for a specific year.

Function Arguments:
- `pdf` (`pd.DataFrame`): the population data in original (raw) format
- `city` (`str`): a string that denotes a city or region (useful for forming the chart title)
- `year` (`str`): the year of the data column to retrieve (also for forming the chart title)
- `k` (`int`): age group interval (i.e., group by every k rows)
- `w` and `h` (`int`): width and height of the Matplotlib figure to plot

Detailed Steps:
- **Transform data**: Call your `transform_data()` function to get the plotting-ready `DataFrame`.
- **Initialize the figure**: Create a new figure with the provided width (`w`) and height (`h`).
- **Plot bars**: Use `Axes.barh()` or `plt.barh()` to create a horizontal bar chart.
  - Plot the female population on the right-hand side.
  - Plot the male population on the left-hand side.

Hints: Call `barh()` twice to plot 'Male' and 'Female' series in the consolidated `DataFrame`. You should study the documentation of the `Axes.barh()` or `plt.barh()` function to understand how to use it, particularly its `width` and `left` arguments.

To achieve the side-by-side effect, you may need to use a negated copy of the 'Male' column values when plotting. In brief, do the following:
- For the 'Female' series, use the 'Age' column as `y` and the 'Female' column as `width`.
- For the 'Male' series, use the 'Age' column as `y`, the 'Male' column as `width` and the negated 'Male' column for `left`.

- **Set the x-axis limits**: Set x-limits to show the bars with some space margin on the left and right edges of the chart.
  - First, find the maximum population value in the `DataFrame` (covering both 'Male' and 'Female' columns).
  - Include a margin, which is 1/10 of the maximum value found. Use the floor division result.
  - This maximum value + margin will be used to set symmetric x-axis limits. The purpose is to make space between the longest bar and the left or right edge on both sides.
  - Set the major ticks on x-axis in units of thousands (showing the letter 'K' in each tick). Note: showing negative x-ticks for male population counts may look weird. Therefore, we will show all ticks as positive.

  Hint: Pass a lambda expression to `set_major_formatter()` to display the x-ticks as absolute values (e.g., showing "50K" instead of "-50K" for the male side). The lambda expression is in the form: `lambda x, _: f'{…}K'` that returns a formatted string that includes the absolute value of x and with letter 'K'.

- **Styling**:
  - Use the blue color #4682b4 for male bars and the red color #ee7a87 for female bars.
  - Label the series to distinguish between "Male" and "Female".
  - Show the legend of "Male" and "Female" (without specifying its location).
  - Set the chart title as "`{city}` *Population Pyramid for Year* `{year}`".
  - Set x-label as "population" and y-label as "age".
  - Apply a tight layout using `tight_layout()` to "auto-fit" everything within the figure area without overlapping or being cut off before returning the `Figure` object.
  - Adopt the default settings for all other styles not mentioned here.

Sample Client Code

```
# Sample client
if __name__ == "__main__":
    try:
        hkpdf, years = read_population_data("q3_data.csv")
        print("Raw data:", hkpdf, "List of years:", years, sep="\n")
    except FileNotFoundError as e:
        print(e)
        sys.exit(1)

    # Plot the default population pyramid for each year
    for year in years:
        pop_fig = population_pyramid(hkpdf, "Hong Kong", year, w=7, h=5)
        pop_fig.savefig(f"hk_{year}_pyramid_k5.png")
```

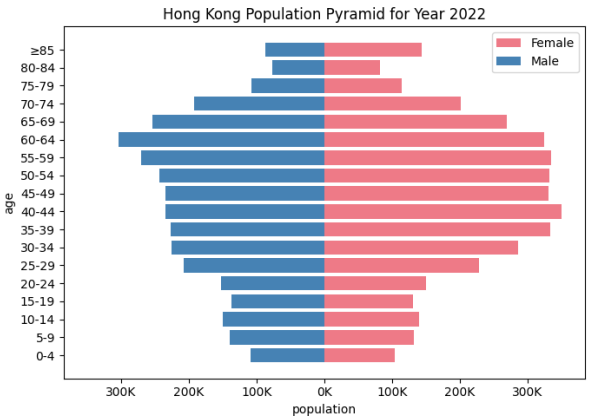
```
    # Plot using other intervals for the age groups
    year = '2025'
    for k in [10, 15, 20]:
        print(f"Year {year} Demographics Grouped by {k} Years:")
        print(transform_data(hkpdf, year, k))
        pop_fig = population_pyramid(hkpdf, "Hong Kong", year, k=k)
        pop_fig.savefig(f"hk_{year}_pyramid_k{k}.png")
```

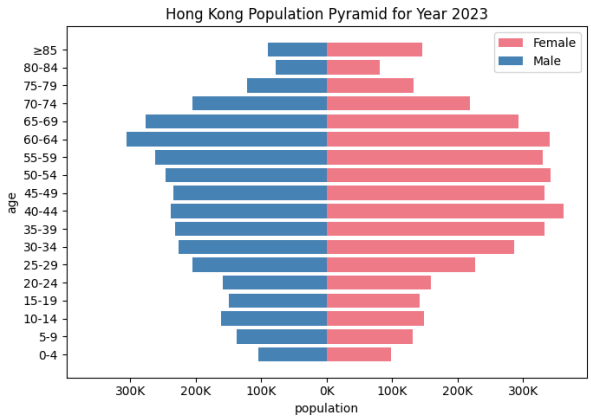
Sample Output

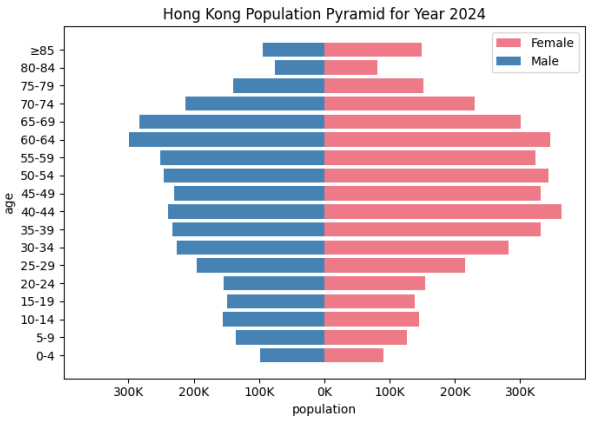
```
Raw data:
        Sex  Age   2022   2023   2024   2025
0      Male    0   17.8   17.0   17.4   16.7
1      Male    1   19.2   18.3   17.4   17.2
2      Male    2   23.3   19.6   18.5   17.6
3      Male    3   24.3   24.1   20.0   18.7
4      Male    4   24.8   25.1   24.4   20.1
..      ...  ...    ...    ...    ...    ...
167  Female   81   15.4   16.0   17.7   17.9
168  Female   82   16.5   15.1   15.6   16.9
169  Female   83   16.6   16.0   14.6   14.9
170  Female   84   17.5   16.1   15.4   13.8
171  Female  ≥85  143.5  146.3  149.2  149.7

[172 rows x 6 columns]
List of years:
['2022', '2023', '2024', '2025']
Year 2025 Demographics Grouped by 10 Years:
      Age   Male  Female
0     0-9  219.5   205.5
1   10-19  293.9   274.4
2   20-29  339.3   353.9
3   30-39  463.8   603.0
4   40-49  473.4   704.4
5   50-59  488.8   664.4
6   60-69  578.2   653.0
7   70-79  373.7   407.9
8  80-≥85  169.5   232.3
Year 2025 Demographics Grouped by 15 Years:
      Age   Male  Female
0    0-14  369.1   345.7
1   15-29  483.6   488.1
2   30-44  705.6   967.9
3   45-59  720.4  1003.9
4   60-74  797.2   892.3
5  75-≥85  324.2   400.9
Year 2025 Demographics Grouped by 20 Years:
      Age   Male  Female
0    0-19  513.4   479.9
1   20-39  803.1   956.9
2   40-59  962.2  1368.8
3   60-79  951.9  1060.9
4  80-≥85  169.5   232.3
```

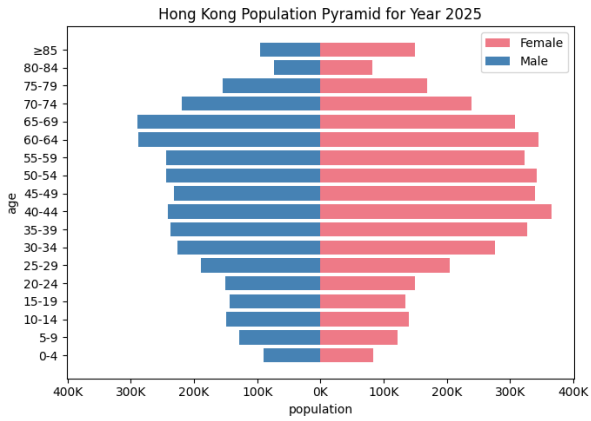
Apart from the above textual output, the sample client code saves the generated figures as PNG files named "`hk_{year}_pyramid_k{k}.png`". You may compare your results with the sample images we provided (see below screenshots) for your testing and debugging. Your output images should match the samples as closely as possible — ideally exactly — including bar lengths, tick marks, and overall styling.

Results for k = 5 and year = '2022', '2023', '2024', '2025':

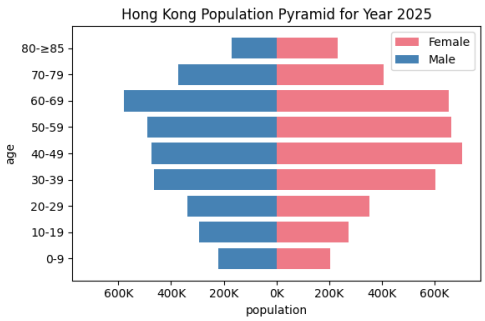


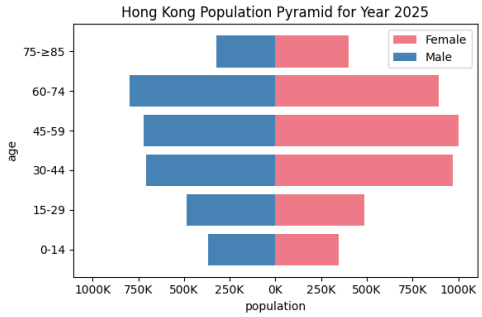


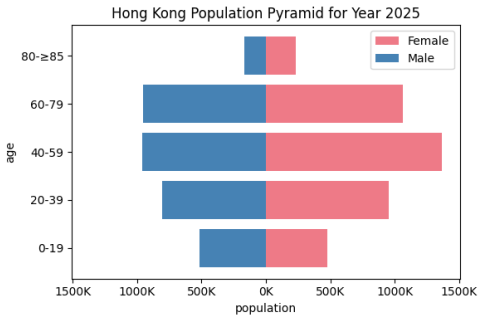




Results for k = 10, 15, 20 and year = '2025':







A simple conclusion drawn from these figures is that female population exceeds the male population in most age groups.

(For your reference, these figures are generated by Matplotlib 3.10.8. Other versions should also work.)